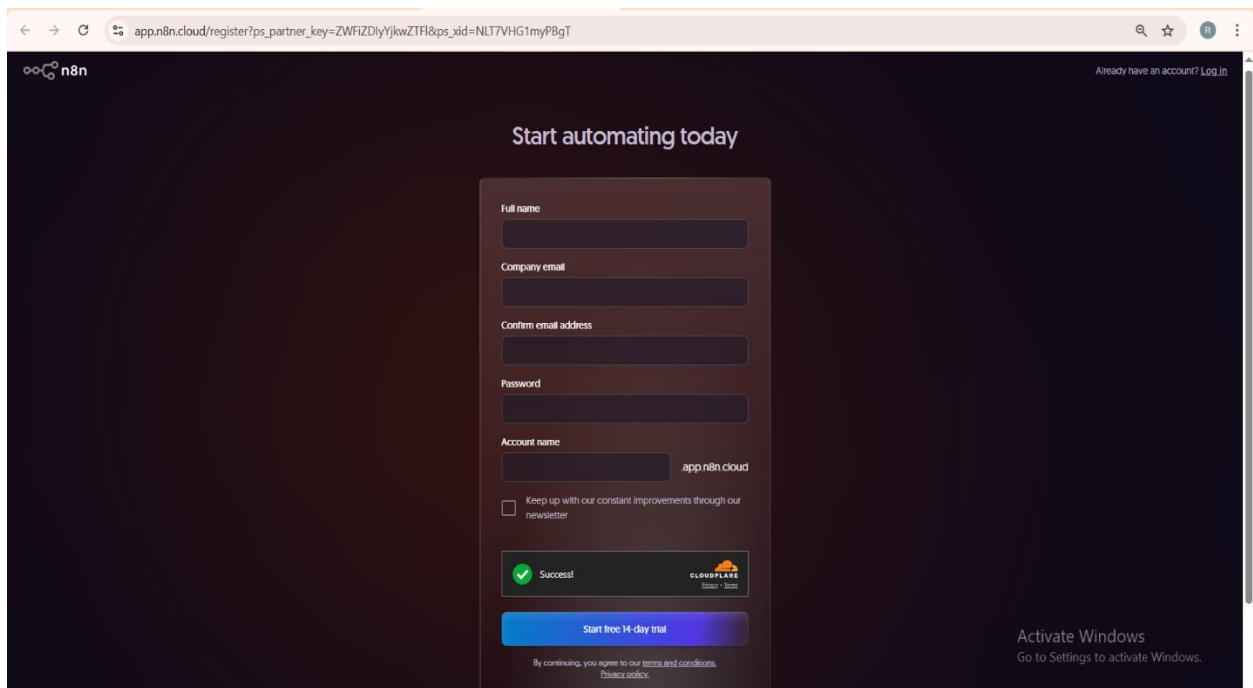
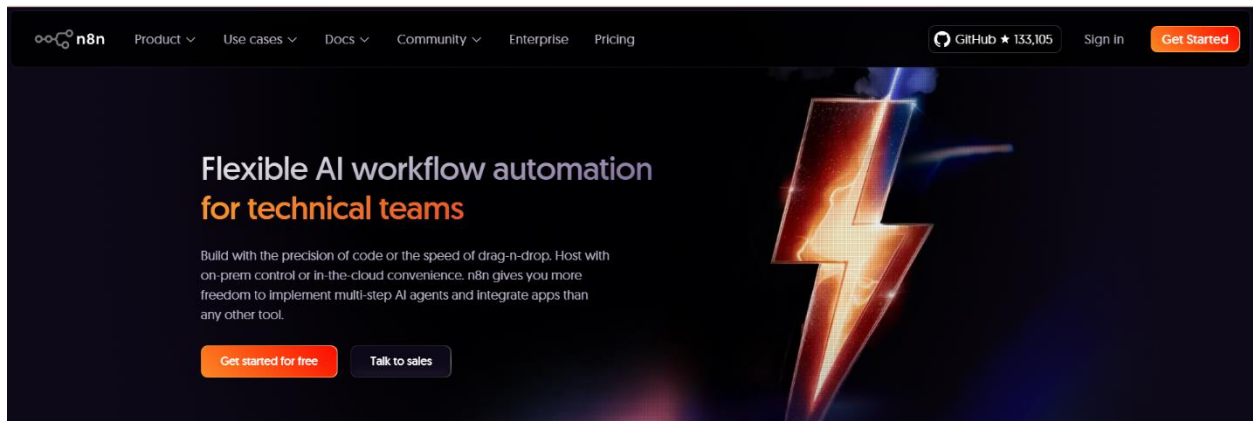


AI AGENTS - Lab Manual

PART A: N8N: Workflow Automation Tool

Link: <https://app.n8n.cloud/login>



Start automating today

Full name

Nikhar Azhar

Company email

rahkinrahza@gmail.com

Confirm email address

rahkinrahza@gmail.com

Password

••••••••

Account name

nikhar

.app.n8n.cloud

☒ Keep up with our constant improvements through our newsletter

✓ Success!


Privacy • Terms

Start free 14-day trial

By continuing, you agree to our [terms and conditions](#) and [Privacy policy](#).

Hi there, Nikhar

Answer these questions to get the most out of your n8n trial

1- What is the size of your company?*

This helps us to customize your experience in n8n

A Only me

B <20

C 20-99

D 100-499

E 500-999

F 1000+

G I'm not using n8n for work

OK

Hi there, Nikhar

Answer these questions to get the most out of your n8n trial

4- Which of these tools have you used before?*

Choose as many as you like

A Zapier

B Make

C n8n

D None

E Other

OK

Invite team members to your workspace

Collaborate on workflows and share app credentials with workspace users

Email

name@email.com

Add another user +

Skip

Invite

app.n8n.cloud/account/launching

n8n

Your workspace is ready!

Start automating

Get started faster: key concepts of n8n



14 days left in your n8n trial0/1000 Executions

Upgrade now

n8n

+

Overview

Personal

Projects

+ Add project

Admin Panel

Templates

Variables

Insights

Help

What's New

Nikhar Azhar

Overview

All the workflows, credentials and executions you have access to

Create Workflow

Prod. executions Last 7 days 0	Failed prod. executions Last 7 days 0	Failure rate Last 7 days 0%	Time saved Last 7 days --	Run time (avg.) Last 7 days 0s
--------------------------------------	---	-----------------------------------	---------------------------------	--------------------------------------

Workflows

Credentials

Executions

Welcome Nikhar!
Create your workflow

Start from scratch

Test a simple AI Agent example

Activate Windows
Go to Settings to activate Windows.

14 days left in your n8n trial0/1000 Executions

Upgrade now

n8n

+

Overview

Personal

Projects

+ Add project

Admin Panel

Templates

Variables

Insights

Help

What's New

Nikhar Azhar

Personal / My workflow + Add tag

Editor

Executions

Evaluations

Inactive

Share

Save

Star 133,112

+
Add first step...

+
Add step...

+
Add step...

+
Add step...

Logs

14 days left in your n8n trial | 0/1000 Executions

Upgrade now

n8n

Personal / My workflow + Add tag

Overview

Personal

Projects

+ Add project

Admin Panel

Templates

Variables

Insights

Help

Editor

Executions

Evaluations

Inactive

Share

Save

...

Star

133,112

+

Add first step...

What triggers this workflow?

A trigger is a step that starts your workflow

Search nodes...

Trigger manually

Runs the flow on clicking a button in n8n. Good for getting started quickly

On app event

Runs the flow when something happens in an app like Telegram, Notion or Airtable

On a schedule

Runs the flow every day, hour, or custom interval

On webhook call

Runs the flow on receiving an HTTP request

On form submission

Generate webforms in n8n and pass their responses to the workflow

Example Workflow:

Personal / My workflow + Add tag

Editor

Executions

Evaluations

Publish

Save

...

+

Q

D

+

+

Q

D

+

1 item

1 item

1 item

1 item

⚡

Gmail Trigger

Gmail Trigger

⚙️

If

+

Add label to message

addLabels: message

Execute workflow

Fixed Expression

Show a node parameter here, to iterate easily

For example, keep your prompt always visible so you can run the workflow while tweaking it

First Node (Gmail):

The screenshot shows the configuration for a 'Gmail Trigger' node. The 'Parameters' tab is active, showing a 'Gmail account' dropdown, 'Poll Times' set to 'Every Minute', and an 'Event' of 'Message Received'. The 'Simplify' toggle is turned on. The 'OUTPUT' tab shows a JSON payload for a received email. A red button labeled 'Fetch Test Event' is visible in the top right of the configuration area.

Parameters Settings **Fetch Test Event**

Credential to connect with
Gmail account

Poll Times
Mode
Every Minute
Add Poll Time

Event
Message Received

Simplify
☒

Filters
No properties
Add Filter

When will this node trigger my flow? ▾

I wish this node would...

OUTPUT 1 item

```
{
  "id": "19bd5ec174e0f49e",
  "threadId": "19bd5bb47eb6841e",
  "snippet": "test2 On Mon, Jan 19, 2026 at 3:09 PM Noor ulAin <noor.ulain@isb.nu.edu.pk>; wrote: this is the test email for ai agent.",
  "payload": {
    "mimeType": "multipart/alternative"
  },
  "sizeEstimate": 7936,
  "historyId": "2750111",
  "internalDate": "1768820565000",
  "labels": [
    {
      "id": "INBOX",
      "name": "INBOX"
    },
    {
      "id": "IMPORTANT",
      "name": "IMPORTANT"
    },
    {
      "id": "CATEGORY_PERSONAL",
      "name": "CATEGORY_PERSONAL"
    }
  ]
}
```

Second Node (IF):

The screenshot shows the configuration for an 'IF' node. The 'Parameters' tab is active, showing a condition: 'fx {{{json.From}}} noor.ulain@isb.nu.edu.pk' with the operator 'contains'. The 'Simplify' toggle is turned on. The 'OUTPUT' tab shows the JSON payload for the 'True Branch (1 item)'. A green arrow points from the 'fx' field in the condition to the 'From' field in the JSON payload.

INPUT 1 item

Parameters Settings **Execute step**

Conditions
fx {{{json.From}}} noor.ulain@isb.nu.edu.pk to T contains
AND
value1 value2 T is equal to
Add condition

Convert types where required
☒

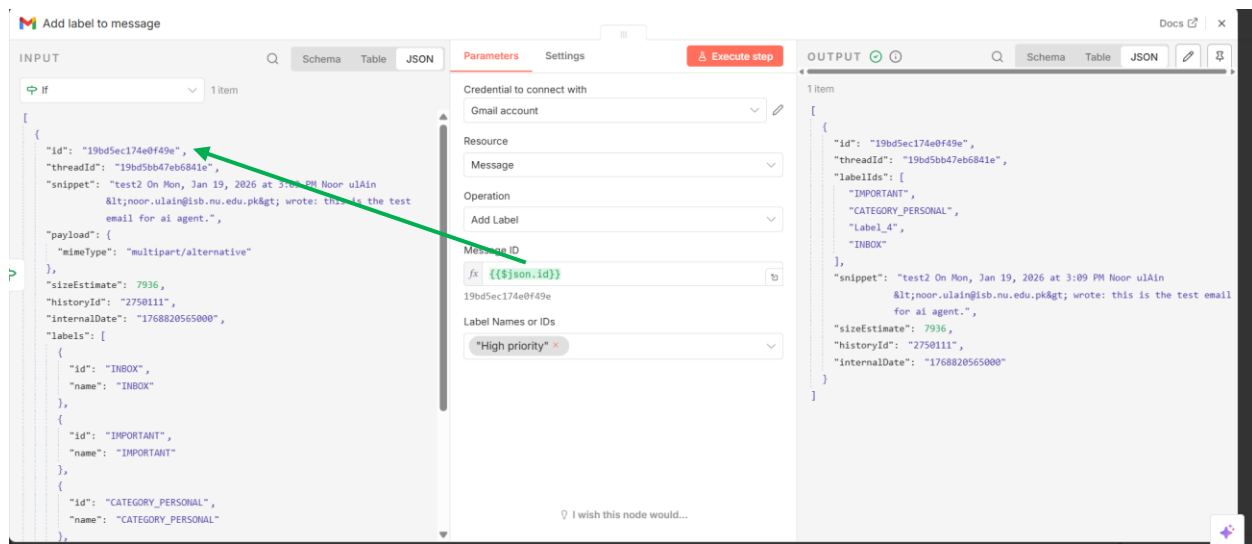
Options
No properties
Add option

I wish this node would...

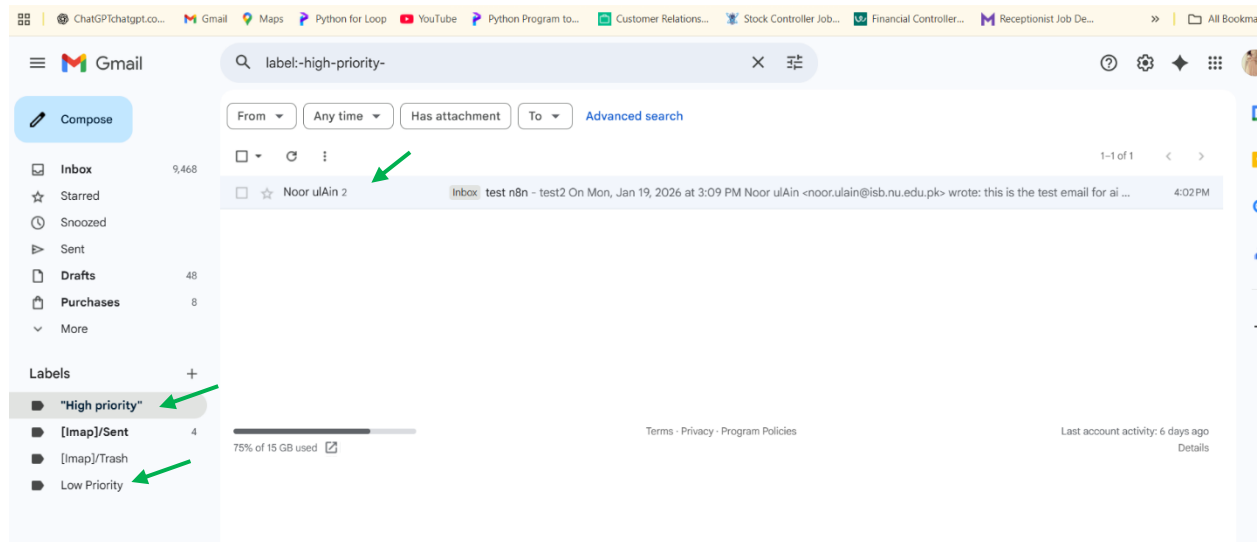
OUTPUT True Branch (1 item) False Branch

```
{
  "id": "19bd5ec174e0f49e",
  "threadId": "19bd5bb47eb6841e",
  "snippet": "test2 On Mon, Jan 19, 2026 at 3:09 PM Noor ulAin <noor.ulain@isb.nu.edu.pk>; wrote: this is the test email for ai agent.",
  "payload": {
    "mimeType": "multipart/alternative"
  },
  "sizeEstimate": 7936,
  "historyId": "2750111",
  "internalDate": "1768820565000",
  "labels": [
    {
      "id": "INBOX",
      "name": "INBOX"
    },
    {
      "id": "IMPORTANT",
      "name": "IMPORTANT"
    },
    {
      "id": "CATEGORY_PERSONAL",
      "name": "CATEGORY_PERSONAL"
    }
  ]
}
```

Third Node (Gmail (Applying Label(s))):



RESULT:



Workflow: Gmail → Assign High / Medium / Low Priority Labels

Scenario

- Emails from **noor.ulain@isb.nu.edu.pk** → High Priority
- Emails from **abc.ai@gmail.com** → Low Priority
- Emails from **someone else** → Medium Priority

Step 1: Gmail Trigger Node

Purpose: Watch for new emails.

1. Node: **Gmail Trigger**
2. Event: **New Email**
3. Authentication: Your Gmail account
4. Output includes:

```
{  
  "id": "17453",  
  "from": " noor.ulain@isb.nu.edu.pk ",  
  "subject": "Project Update",  
  "body": "Please check the new report"  
}
```

Step 2: Set Node (Map Email to Priority)

Purpose: Assign a priority based on sender email.

1. Add **Set Node** after Gmail Trigger
2. Add a field: priority
3. Use **expression**:

```
{{ $json["from"].includes("noor.ulain@isb.nu.edu.pk ") ? "High" :  
($json["from"].includes("abc.ai@gmail.com") ? "Low" : "Medium") }}
```

This will dynamically set the priority for the email.

Step 3: Check / Create Labels (Optional)

Gmail needs **labels created first** to apply them. You have two options:

Option A: Pre-create labels in Gmail (Recommended as we need it only once)

- Go to Gmail → Settings → Labels → Create:
 - High Priority
 - Medium Priority
 - Low Priority

Option B: Create labels dynamically in n8n

1. Add **Gmail Node** → Operation: **Create Label**
2. Expression for **Label Name**:
`{{ $json["priority"] }}` Priority
3. Connect it **before applying label**
4. Use IF Node or error handling to **skip if label exists**

Note: Dynamic creation is optional. Usually simpler to pre-create labels.

Step 4: Gmail Node (Apply Label / Star / Important)

1. Add **Gmail Node** → Operation: **Modify Email**
2. Input: Message ID from Gmail Trigger → `{{ $json["id"] }}`
3. Apply **Label** → Expression:
`{{ $json["priority"] }}` Priority
4. Optional: Star / Mark Important if desired

Step 5: Testing

- Send email from noor.ulain@isb.nu.edu.pk → Label applied = High Priority
- Send email from abc.ai@gmail.com → Label applied = Low Priority
- Send from any other → Label applied = Medium Priority

PART B: PYTHON IMPLEMENTATION

Agents & environments

An agent can be implemented as a function/class that maps percepts (observations) to actions, while the environment updates its internal state after actions.

Practice snippet (agent + environment loop)

Practice: basic agent-environment loop (edit observations/actions as needed)

for t **in** range(5):

```
percept = get_percept_somewhat()    # TODO: you define percept format
action = agent_program(percept)      # TODO: your decision rule
apply_action_somewhat(action)        # TODO: environment transition
```

Practical PEAS (with Python structure)

PEAS = Performance measure, Environment, Actuators, Sensors, use it to write a task description before coding.

PEAS as a dictionary

```
PEAS = {  
    "Performance": ["accuracy", "time", "energy"], # TODO: choose measures  
    "Environment": ["grid", "rooms", "web"],      # TODO  
    "Actuators": ["move_left", "move_right"],     # TODO  
    "Sensors": ["position_sensor", "dirt_sensor"] # TODO  
}
```

```
print(PEAS)
```

Environment types (how to model in code)

“Artificial Intelligence: A Modern Approach” classifies environments by properties like observable, deterministic, episodic, static, discrete, single/multi-agent.

```
env_type = {  
    "observable": "full",    # full / partial  
    "deterministic": True,   # True / False  
    "episodic": False,      # True / False  
    "static": True,         # True / False  
    "discrete": True,       # True / False  
    "agents": 1             # 1 or many  
}
```

```
print(env_type)
```

Agent types

“Artificial Intelligence: A Modern Approach” lists basic agent architectures: simple reflex, reflex with state, goal-based, utility-based (and learning variants).

1) Simple reflex agent (condition → action)

This is “if percept says X, do Y” (no memory).

```

def reflex_agent(percept):
    # percept could be (location, status) or any structure you define
    location, status = percept

    if status == "Dirty":
        return "SUCK"
    elif location == "A":
        return "RIGHT"
    else:
        return "LEFT"

```

(Demonstrating if/elif/else **rule mapping**.)

2) Reflex agent with state (memory)/ Model based Agent

Keeps internal variables and updates them each step (use class attributes).

```

class ReflexWithState:
    def __init__(self):
        self.memory = {"A": None, "B": None}  # internal model/belief

    def act(self, percept):
        location, status = percept
        self.memory[location] = status        # update memory

        # TODO: write rule that uses self.memory + percept
        return "SOME_ACTION"

```

3) Goal-based agent (uses a goal test)

Goal-based agents choose actions that move toward a goal condition.

```

class GoalBasedAgent:
    def __init__(self, goal):

```

```
self.goal = goal
```

```
def act(self, state):  
    if state == self.goal:  
        return "STOP"  
    # TODO: choose action that reduces distance to goal  
    return "MOVE"
```

4) Utility-based agent (score actions)

Utility-based agents evaluate possible actions and pick the best.

```
def choose_action(actions, utility_fn):  
    scored = [(a, utility_fn(a)) for a in actions]  
    return max(scored, key=lambda x: x[1])[0] # best action
```

Deterministic vs stochastic

Deterministic: For a given current state and action, the next state is always the same (no randomness), so repeating the same action sequence gives the same trajectory.

Stochastic: For a given current state and action, the next state is not guaranteed, outcomes are probabilistic (randomness/uncertainty), so repeating the same action sequence can produce different trajectories.

```
import random  
  
def transition(current_state, intended_action, p_success=0.8):  
    if random.random() < p_success:  
        return ("next_state_intended", intended_action)  
    else:  
        return ("next_state_random", "RANDOM_EFFECT")
```

This demonstrates the syntax for probabilistic outcomes.